VERONICA: A Runtime Containment Layer for Compositional Resource Amplification in LLM Agent Systems

VERONICA: A Runtime Containment Layer for Compositional Resource Amplification in LLM Agent Systems

Author: amabito **Version:** 3.4.2 | **Date:** 2026-03-09 | **Status:** Working Paper **Source:** https://github.com/amabito/veronica-core **Package:** https://pypi.org/project/veronica-core/

Abstract

Large language model (LLM) agent frameworks expose a class of runaway failure modes that differ qualitatively from classical software defects. Retry amplification, recursive tool invocation loops, multi-agent cost cascades, and WebSocket session runaways share a common structure: they originate from individually reasonable local decisions that compose into unbounded global resource consumption. Existing frameworks (LangChain, AG2, LangGraph) delegate containment to application developers through per-call timeouts and manual budget checks, leaving compositional failures unaddressed.

We present VERONICA (Verified Execution Runtime for Observably Networked Intelligent Contained Agents), a runtime containment layer that enforces chain-level resource bounds without requiring modifications to agent logic. VERONICA introduces four orthogonal primitives – `BudgetEnforcer`, `AgentStepGuard`, `CircuitBreaker`, and `RetryContainer` – unified under a `RuntimePolicy` structural protocol and composed by `PolicyPipeline` with first-denial-wins semantics. A chain-scoped `ExecutionContext` enforces simultaneous bounds on cost, step count, retry count, and wall-clock time across all LLM and tool calls within a single agent run, with propagation through ASGI/WSGI middleware and WebSocket sessions.

We present four formal safety guarantees (G1–G6), benchmark results demonstrating 83–90% operation reduction versus uncontained baselines, and a policy pipeline overhead of 11.43 microseconds per call. VERONICA is available as `veronica-core` on PyPI (v3.4.2, 4844 tests, 94% coverage).

---

1. Introduction

LLM agent systems execute multi-step plans by repeatedly calling language models and invoking tools based on model outputs. This structure creates several runaway failure modes that are absent in classical request/response software:

Retry amplification. When L nested tool layers each retry r times on failure, a single user action generates at most (1 + r)^L LLM calls. At L=3, r=3, this is 64 calls from a single user action. Frameworks that implement per-call retry budgets do not track cross-layer retry counts; the ceiling is the product of all per-layer budgets.

Recursive tool loops. An agent producing a tool call whose output triggers the same tool call will loop indefinitely unless an external step counter intervenes. LangChain's `max_iterations` parameter addresses this within a single chain but does not apply across chain boundaries in multi-agent topologies.

Multi-agent amplification. A coordinator spawning K sub-agents in a tree of depth D creates (K^(D+1) - 1) / (K - 1) total agents; at K=5, D=2, that is 31 agents. If each agent independently makes S steps with r retries, total calls are 31 * S * (1 + r). No existing framework enforces a cross-agent cost ceiling without custom application code.

WebSocket runaway. Long-running WebSocket sessions that stream LLM output accumulate cost and step counts across hundreds of receive/send cycles. Existing ASGI frameworks provide no budget enforcement for WebSocket scopes.

These failure modes are compositional: they arise from combining features (retry logic, agent spawning, tool invocation) that are individually reasonable but collectively unbounded. Classical per-call defenses are insufficient because each call is legitimate; only the aggregate violates the resource constraint.

VERONICA addresses these failure modes through a single architectural decision: containment is enforced at the chain level rather than the call level. Each agent run or HTTP request receives a dedicated `ExecutionContext` that tracks cumulative cost, step count, and retry count across all operations, enforcing hard limits without requiring agent-level awareness of the containment layer.

1.1 Contributions

1. A formal model of LLM agent resource amplification and its compositional structure (Section 3).
2. A runtime containment system with four orthogonal primitives composable via `PolicyPipeline` (Section 4).

3. Six formal safety guarantees (G1–G6) with proofs grounded in code paths (Section 5).
4. Empirical evaluation against uncontained baselines showing 53–90% operation reduction with sub-12-microsecond overhead per call (Section 6).
5. An open-source implementation (`veronica-core`, PyPI) with 4844 tests and 94% coverage (Section 7).

---

2. Background and Related Work

2.1 LLM Agent Frameworks

LangChain [Harrison et al., 2022] provides `max_iterations` for chain step limits and per-call retry logic through `tenacity`-backed decorators. It does not enforce cross-chain cost ceilings, multi-agent retry budgets, or WebSocket containment.

AG2 (AutoGen) [Wu et al., 2023] supports nested agent conversations with configurable `max_consecutive_auto_reply`. Resource limits are delegated to application code. No circuit breaker or distributed budget enforcement is provided.

LangGraph [LangChain, 2024] provides graph-based agent orchestration with checkpoint support and step-level event streams. Budget enforcement is not a first-class primitive; users must instrument graph nodes manually.

CrewAI [Moura, 2024] provides role-based agent teams with task delegation. Cost tracking is not provided at the framework level.

2.2 Containment and Safety Libraries

Guardrails AI focuses on output validation: schema enforcement, content filtering, and structured output parsing. It does not address resource consumption during execution.

NeMo Guardrails provides dialog flow control and safety filtering through conversation rail specifications. Retry and cost containment are outside scope.

OpenAI function calling with `tool_choice` parameters constrains which tools the model selects but does not bound total cost or retry depth.

None of the above systems enforce chain-level cost ceilings, cross-layer retry budgets, or circuit breaking at the framework abstraction level.

2.3 Circuit Breakers and Bulkheads

The circuit breaker pattern [Nygard, 2007] is standard in distributed systems for failure isolation. VERONICA adapts this pattern to the LLM call domain, where “failure” means exceeding a resource threshold rather than service unavailability. The HALF_OPEN single-slot constraint follows the original pattern; the Lua-atomic distributed variant extends it to multi-process deployments.

---

3. Amplification Model

3.1 Definitions

Definition 1 (Call Node). A call node v is a single LLM API call or tool invocation with attributes: cost(v) >= 0 USD, tokens_in(v), tokens_out(v), retries(v) >= 0.

Definition 2 (Agent Chain). An agent chain C is a sequence of call nodes v_1, v_2, ⋯, v_n with:

- total_cost(C) = sum_{i=1}^{n} cost(v_i)
- total_steps(C) = n
- total_retries(C) = sum_{i=1}^{n} retries(v_i)

Definition 3 (Amplification Factor). For a chain C with known minimum required steps min_steps(C):

```
A(C) = total_steps(C) / min_steps(C)
```

In pathological cases (retry explosion, recursive loops), A(C) is unbounded.

3.2 Multi-Layer Retry Amplification

Theorem 1. For a nested call structure with L layers, each independently retrying with r_i retries at layer i, the worst-case total calls are:

```
Total_calls = product_{i=1}^{L} (1 + r_i)
```

Proof. By induction on L. For L=1: at most 1 + r_1 calls. Inductively, each call at layer k triggers at most product_{j=k+1}^{L} (1 + r_j) calls in deeper layers; summing over at most (1 + r_k) attempts at layer k gives the product. QED.

Example. At L=3, r_i=3 for all i: Total_calls = 4^3 = 64. At $0.01/call, a single user action incurs $0.64 in the worst case versus $0.01 for the minimum path.

3.3 Multi-Agent Amplification

Theorem 2. For a coordinator spawning K sub-agents to depth D, each making S steps with r retries:

```
Total_calls = ((K^{D+1} - 1) / (K - 1)) * S * (1 + r)
```

Example. At K=5, D=2, S=10, r=3: Total_calls = 31 * 10 * 4 = 1,240.

3.4 VERONICA Containment Bounds

Theorem 3 (Cost Bound). For any chain under an `ExecutionContext` with `max_cost_usd = C`: total_cost(C) <= C, independent of agent count, retry depth, or topology.

Theorem 4 (Step Bound). For any chain under an `ExecutionContext` with `max_steps = S`: total_steps(C) <= S.

Theorem 5 (Retry Bound). For any chain under an `ExecutionContext` with `max_retries_total = R`: total_retries(C) <= R, bounding total calls at 1 + R.

Proofs are given in Section 5 (G1–G3). The bounds hold simultaneously and independently; violation of any triggers `Decision.HALT` for all subsequent calls.

---

4. System Architecture

4.1 Overview

```
User / Framework
      |
      v
 [ExecutionContext]  <-- chain-level containment boundary
      |
      +-- [PolicyPipeline]
      |       |
      |       +-- [BudgetEnforcer]     cost_usd ceiling
      |       +-- [AgentStepGuard]     max_steps limit
      |       +-- [CircuitBreaker]     failure isolation
      |       +-- [RetryContainer]     retry budget
      |
      +-- [ExecutionGraph]             call-tree observation
      |
      +-- [CancellationToken]          cooperative shutdown
      |
      +-- [SharedTimeoutPool]          wall-clock timeout
      |
      v
   LLM API / Tools
```

The containment layer sits between the agent framework and the LLM API. Each `wrap_llm_call()` or `wrap_tool_call()` invocation passes through the policy pipeline before dispatching to the underlying callable.

4.2 RuntimePolicy Protocol

All policy primitives implement the `RuntimePolicy` structural protocol (`src/veronica_core/runtime_policy.py`):

```
@runtime_checkable
class RuntimePolicy(Protocol):
```

```
    def check(self, context: PolicyContext) -> PolicyDecision: ...
    def reset(self) -> None: ...

    @property
    def policy_type(self) -> str: ...
```

`PolicyContext` carries ambient information about the current operation:

```
@dataclass
class PolicyContext:
    cost_usd: float = 0.0
    step_count: int = 0
    entity_id: str = ""
    chain_id: str = ""
    timestamp: float = field(default_factory=time.time)
    metadata: dict = field(default_factory=dict)
```

`PolicyDecision` carries the allow/deny result, policy identity, and optional degradation actions (model downgrade, rate limiting):

```
@dataclass
class PolicyDecision:
    allowed: bool
    policy_type: str
    reason: str = ""
    partial_result: Any = None
    degradation_action: str | None = None
    fallback_model: str | None = None
    rate_limit_ms: int = 0
```

The `degradation_action` field enables graceful degradation responses (model downgrade, context trimming, rate limiting) as alternatives to hard denial, allowing callers to adapt behavior without necessarily halting.

4.3 PolicyPipeline

`PolicyPipeline` (`src/veronica_core/runtime_policy.py`) composes multiple `RuntimePolicy` instances with AND semantics: the first denial terminates evaluation and returns the denying decision.

```
class PolicyPipeline:
    def evaluate(self, context: PolicyContext) -> PolicyDecision:
        for policy in self._policies:
            decision = policy.check(context)
            if not decision.allowed:
                return decision  # First denial wins
        return PolicyDecision(allowed=True, policy_type="pipeline", ...)
```

No override mechanism exists. This is a deliberate design decision: safety policies must not be overridable by lower-priority policies or application code.

4.4 BudgetEnforcer

`BudgetEnforcer` (`src/veronica_core/budget.py`) enforces a USD ceiling with thread-safe check-before-commit semantics:

```
def spend(self, amount_usd: float) -> bool:
    with self._lock:
        projected = self._spent_usd + amount_usd
        if projected > self.limit_usd:
            self._exceeded = True
            return False
        self._spent_usd = projected
        self._call_count += 1
        return True
```

Input validation at both construction (`__post_init__`) and call time (`spend`) rejects NaN, Inf, and negative amounts with `ValueError`, preventing cost poisoning attacks that could corrupt the accumulator via IEEE-754 arithmetic anomalies.

Measured overhead: 0.191 microseconds per `spend()` call (100,000 calls).

4.5 AgentStepGuard

`AgentStepGuard` (`src/veronica_core/agent_guard.py`) limits agent iterations via increment-before-check protocol:

```
def step(self, result: Any = None) -> bool:
    with self._lock:
        self._current_step += 1        # increment first
        if result is not None:
            self._last_result = result # preserve partial result
        if self._current_step >= self.max_steps:
            return False               # then check
    return True
```

The increment-before-check ordering prevents the off-by-one error where `max_steps=25` would allow 26 steps if check-before-increment were used. The `_last_result` field preserves the most recent partial output for halted chains.

4.6 CircuitBreaker

`CircuitBreaker` (`src/veronica_core/circuit_breaker.py`) implements the three-state circuit breaker pattern with atomic state transitions via `threading.Lock`. States:

- **CLOSED**: consecutive failures < threshold; all requests allowed
- **OPEN**: failures >= threshold; all requests denied
- **HALF_OPEN**: recovery timeout elapsed; exactly one test request allowed

The HALF_OPEN single-slot constraint, enforced by `_half_open_in_flight`, prevents thundering-herd on recovery: concurrent callers in HALF_OPEN all receive `allowed=False` except the first. This is

identical in semantics to the original circuit breaker pattern [Nygard, 2007] and prevents load amplification on an already-stressed endpoint.

`bind_to_context()` prevents accidental sharing of a `CircuitBreaker` instance across independent chains, which would corrupt failure counts.

In distributed deployments (`src/veronica_core/distributed.py`), state transitions use Lua scripts executed atomically by Redis `eval()`, extending the consistency guarantee to multi-process environments without application-level locking.

Measured overhead: 0.528 microseconds per `check()` call (100,000 calls, CLOSED state).

4.7 RetryContainer

`RetryContainer` (`src/veronica_core/retry.py`) enforces a chain-wide retry budget with exponential backoff and jitter:

```
def execute(self, fn: Callable[..., T], *args, **kwargs) -> T:
    for attempt in range(self.max_retries + 1):
        try:
            return fn(*args, **kwargs)
        except Exception as e:
            if attempt == self.max_retries:
                raise
            delay = min(self.backoff_base * (2 ** attempt), self.backoff_max)
            delay *= 1.0 + self.jitter * (2 * random.random() - 1)
            time.sleep(delay)
```

Jitter (default +/-25%) prevents thundering-herd when multiple `RetryContainer` instances retry simultaneously after a shared service failure. The serialization lock held during `fn()` execution enforces serial retry semantics, preventing concurrent retries from racing on the attempt counter; this is a deliberate design decision, not a performance issue.

4.8 ExecutionContext

`ExecutionContext` (`src/veronica_core/containment/execution_context.py`) is the chain-level container. `ExecutionConfig` specifies hard limits:

```
@dataclass(frozen=True)
class ExecutionConfig:
    max_cost_usd: float
    max_steps: int
    max_retries_total: int
    timeout_ms: int = 0
```

All limits are validated at construction to be finite and non-negative. The context is used as a context manager:

```
config = ExecutionConfig(
    max_cost_usd=1.0, max_steps=50,
```

```
    max_retries_total=10, timeout_ms=30_000
)
with ExecutionContext(config=config, pipeline=pipeline) as ctx:
    decision = ctx.wrap_llm_call(fn=lambda: client.chat(...))
    if decision == Decision.HALT:
        break
```

On entering the context manager, the wall-clock timeout is registered with `SharedTimeoutPool` (a singleton daemon-thread scheduler with a heap-priority queue of deadline-callback pairs). When the deadline fires, `CancellationToken.cancel()` is called, and all subsequent `wrap_llm_call()` invocations return `Decision.HALT` without dispatching.

Measured overhead: 11.43 microseconds per `wrap_llm_call()` call over a full `ExecutionContext` round-trip (10,000 calls).

4.9 Middleware Integration

`VeronicaASGIMiddleware` and `VeronicaWSGIMiddleware` (`src/veronica_core/middleware.py`) create a fresh `ExecutionContext` per HTTP request, stored in a `ContextVar` accessible via `get_current_execution_context()`. For WebSocket scopes, each `receive()` and `send()` call increments the step counter. Budget exceeded at pre-flight returns HTTP 429; mid-session limit triggers `websocket.close` code 1008.

Integration with LangChain, AG2, LangGraph, LlamaIndex, CrewAI, and ROS2 is provided through framework-specific adapters in `src/veronica_core/adapters/`.

4.10 ExecutionGraph

`ExecutionGraph` (`src/veronica_core/containment/execution_graph.py`) tracks the parent-child call tree for observation and diagnostics. Each LLM and tool call creates a `Node` with timing, cost, and token counts. The graph supports divergence heuristics based on repeated `(kind, name)` node signatures, enabling early detection of pathological loops before they exhaust the budget.

---

5. Formal Safety Guarantees

We state six safety guarantees and identify the code paths that enforce each. All guarantees assume a correctly configured `ExecutionContext` with finite, non-negative limit values; G6 proves that invalid configurations cannot be created.

G1: Cost Bound

Guarantee. For any agent chain under an `ExecutionContext` with `ExecutionConfig.max_cost_usd = C`:

```
total_cost(chain) <= C
```

Proof. The budget accumulator is `_cost_usd_accumulated` in `ExecutionContext`. Every LLM call passes through `_wrap()`, which calls `_commit_cost(actual_cost)` after the callable returns:

```
with self._lock:
    projected = self._cost_usd_accumulated + actual_cost
    if projected > self._config.max_cost_usd:
        self._aborted = True
        self._abort_reason = "budget_exceeded"
        return Decision.HALT
    self._cost_usd_accumulated = projected
```

No code path commits cost without first checking the ceiling. The lock prevents concurrent commits from racing past the ceiling. Additionally, `BudgetEnforcer.spend()` (`src/veronica_core/budget.py`) uses the identical pattern for standalone use.

Input validation in `BudgetEnforcer.__post_init__()` and `spend()` rejects NaN, Inf, and negative values, preventing cost poisoning via IEEE-754 arithmetic anomalies.

In distributed mode (`RedisBudgetBackend`, `src/veronica_core/distributed.py`), Redis `INCRBYFLOAT` is atomic; an epsilon guard (`_BUDGET_EPSILON = 1e-9`) prevents spurious under-enforcement from floating-point rounding.

G1 holds in both local and distributed configurations. QED.

G2: Termination Guarantee

Guarantee. For any agent chain under an `ExecutionContext` with `ExecutionConfig.max_steps = S`:

```
successful_operations(chain) <= S
```

Proof. The step counter is `_step_count` in `ExecutionContext`. Every call through `wrap_llm_call()` or `wrap_tool_call()` passes through `_wrap()`:

```
with self._lock:
    if self._step_count >= self._config.max_steps:
        self._aborted = True
        return Decision.HALT
```

After a successful call, `_step_count` is incremented. The check-before-increment ordering means: at step count S, the next call is denied before `_step_count` would become S+1. No code path increments past S. QED.

G3: Retry Budget

Guarantee. For any agent chain under an `ExecutionContext` with `ExecutionConfig.max_retries_total = R`:

```
total_retries(chain) <= R
```

Proof. The retry counter is `_retries_used` in `ExecutionContext`. Before each retry in `_wrap()`, `_check_retry_budget()` is called:

```
with self._lock:
    if self._retries_used >= self._config.max_retries_total:
        self._aborted = True
        return Decision.HALT
    self._retries_used += 1
```

The check-before-increment protocol provides the same bound as G2. When `RetryContainer` instances operate within an `ExecutionContext`, the chain-level check fires before each retry attempt, enforcing the cross-container ceiling. QED.

G4: Failure Isolation

Guarantee. Once `CircuitBreaker` enters OPEN state:

```
state == OPEN => check(ctx).allowed == False
```

during all calls until at least `recovery_timeout` seconds have elapsed.

Proof. State transitions are protected by `threading.Lock`. The only transition out of OPEN is OPEN -> HALF_OPEN, gated by `_maybe_half_open_locked()`:

```
time.time() - self._last_failure_time >= self.recovery_timeout
```

evaluated under lock, preventing two threads from simultaneously observing OPEN as expired. No code path transitions OPEN directly to CLOSED; the mandatory HALF_OPEN state ensures at least one test call before recovery.

In distributed mode, Lua scripts executed via Redis `eval()` provide identical atomicity across processes. QED.

G5: Wall-Clock Timeout

Guarantee. For any chain under `ExecutionContext` with `timeout_ms = T > 0`:

```
elapsed_ms > T => all new wrap_llm_call() return Decision.HALT
```

Proof. On `__enter__`, a deadline is registered with `SharedTimeoutPool`. When the deadline fires, `_on_timeout()` calls `CancellationToken.cancel()`. All subsequent `wrap_llm_call()` calls check `CancellationToken.is_cancelled` before dispatching; cancelled contexts return `Decision.HALT` immediately.

Caveat. Operations already in-flight when the timeout fires may complete before observing cancellation. G5 applies only to new operations; in-flight operations must poll `is_cancelled` for preemptive cancellation. QED.

G6: Configuration Validity

Guarantee. An `ExecutionContext` with NaN, Inf, or negative limits cannot be created.

Proof. `ExecutionConfig.__post_init__()` validates all fields:

```
if math.isnan(self.max_cost_usd) or math.isinf(self.max_cost_usd):
    raise ValueError(...)
if self.max_cost_usd < 0:
    raise ValueError(...)
# ... analogous for max_steps, max_retries_total, timeout_ms
```

`BudgetEnforcer.__post_init__()`, `CircuitBreaker.__post_init__()`, and `AgentStepGuard` validation analogously reject invalid inputs. Invalid configurations raise `ValueError` at construction, not at runtime. QED.

Compositional Safety

Theorem 6 (Compositional Bound). For a `PolicyPipeline` containing `BudgetEnforcer(limit_usd=C)`, `AgentStepGuard(max_steps=S)`, and `RetryContainer(max_retries=R)`, the following hold simultaneously:

- total_cost <= C
- total_steps <= S
- total_retries <= R

Proof sketch. Each bound is enforced by an independent check-before-commit protocol under its own lock. Violation of any bound returns `False`/`HALT` before the underlying callable is invoked. `PolicyPipeline.evaluate()` returns the first denial, preventing execution from reaching any primitive with a satisfied check when an earlier primitive has already denied. No code path bypasses any check. QED.

---

6. Evaluation

All benchmarks use stub LLM/tool implementations with no network calls. Experiments run on a single host (no distributed backend). Source: `benchmarks/` in the repository.

6.1 Benchmark Scenarios

We evaluate four failure modes corresponding to Section 1:

Scenario	Baseline Setup	VERONICA Config
Retry amplification	3 layers x 3 retries = 27 calls	`max_retries_total=5`
Recursive tools	20 recursive tool calls	`max_steps=5`
Multi-agent loop	Planner/critic loop, 30 iterations	`AgentStepGuard(max_steps=8)`
WebSocket runaway	50 send/receive pairs = 100 ops	`max_steps=10`

6.2 Results

Scenario	Baseline Ops	VERONICA Ops	Reduction	Containment Mechanism
Retry amplification	27	3	88.9%	`max_retries_total`, `RetryContainer`
Recursive tools	20	5	75.0%	`max_steps`, `wrap_tool_call()`
Multi-agent loop	60	28	53.3%	`AgentStepGuard(max_steps=8)`
WebSocket runaway	100	10	90.0%	`max_steps`, close code 1008

The retry amplification result confirms Theorem 5: 3-layer x 3-retry = 27 theoretical calls is bounded to 3 actual calls (max_retries_total=5 limits to 1+5=6 calls, but the stub LLM succeeds after 2 failures, so 3 tasks x 1 call each = 3).

The WebSocket scenario shows containment latency of 0.0168 ms from limit detection to `websocket.close` code 1008, consistent with sub-millisecond cooperative shutdown.

6.3 Policy Pipeline Overhead

Primitive	Measurement	Calls
`BudgetEnforcer.spend()`	0.191 us/call	100,000
`CircuitBreaker.check()`	0.528 us/call	100,000
`ExecutionContext.wrap_llm_call()` (full round-trip)	11.43 us/call	10,000

The full `ExecutionContext` round-trip includes: `CancellationToken` check, budget pre-flight, `PolicyPipeline.evaluate()`, callable dispatch (stub returning `None`), cost commit, step increment, and `ExecutionGraph` node creation. At 11.43 us/call, the overhead is dominated by lock acquisition and `ExecutionGraph` node allocation. For LLM API calls with typical latency of 500–5000 ms, this overhead is less than 0.002% of total call time.

6.4 Comparison with Existing Frameworks

Feature	VERONICA	LangChain	AG2	LangGraph
Chain-level cost ceiling	Yes (`max_cost_usd`)	No	No	No
Cross-layer retry budget	Yes (`max_retries_total`)	No	No	No

Feature	VERONICA	LangChain	AG2	LangGraph
Circuit breaker	Yes (`CircuitBreaker`)	No	No	No
WebSocket containment	Yes (ASGI middleware)	No	No	No
Step limit	Yes (`AgentStepGuard`)	Yes (`max_iterations`)	Yes	Yes
Multi-agent budget	Yes (Redis backend)	No	No	No
Cost poisoning defense	Yes (NaN/Inf validation)	No	No	No
Partial result preservation	Yes (`PartialResultBuffer`)	No	No	No
Formal safety guarantees	Yes (G1–G6)	No	No	No
Framework-agnostic	Yes	–	–	–

VERONICA is orthogonal to all compared frameworks: it operates at the resource consumption layer rather than the content or dialog layer, and can be composed with any of them through the adapter modules in `src/veronica_core/adapters/`.

---

7. Implementation

VERONICA is implemented in Python 3.10+. The library is structured to be framework-agnostic with no hard dependency on any specific LLM client or agent framework.

Core dependencies:

- `threading` (standard library): lock-based concurrency for all state-machine primitives
- `contextvars` (standard library): per-request `ContextVar` storage for middleware integration
- `redis` (optional): distributed budget backend and atomic Lua scripts for `DistributedCircuitBreaker`
- `opentelemetry-api` (optional): `OTelExecutionGraphObserver` for execution graph export

Source structure:

```
src/veronica_core/
  runtime_policy.py          # RuntimePolicy, PolicyContext, PolicyDecision, PolicyPipeline
  budget.py                  # BudgetEnforcer
  agent_guard.py             # AgentStepGuard
  circuit_breaker.py         # CircuitBreaker (local + distributed)
  retry.py                   # RetryContainer
  distributed.py             # RedisBudgetBackend, DistributedCircuitBreaker
  containment/
    execution_context.py     # ExecutionContext, ExecutionConfig, WrapOptions
    execution_graph.py       # ExecutionGraph, Node
    timeout_pool.py          # SharedTimeoutPool, CancellationToken
```

```
    budget_allocator.py      # BudgetAllocator (hierarchical budgets)
  middleware.py              # VeronicaASGIMiddleware, VeronicaWSGIMiddleware
  adapters/
    langchain.py             # LangChain callback adapter
    ag2.py                   # AG2 capability adapter
    langgraph.py             # LangGraph node wrapper
    mcp.py                   # MCP containment adapter (sync)
    mcp_async.py             # MCP containment adapter (async)
    crewai.py                # CrewAI adapter
    llamaindex.py            # LlamaIndex adapter
    ros2.py                  # ROS2 adapter
  partial.py                 # PartialResultBuffer
  otel.py                    # OTelExecutionGraphObserver
```

Distribution: `veronica-core` on PyPI (v3.4.2).

Testing: 4844 tests, 94% coverage (v3.4.2). Test categories: unit tests for each primitive, integration tests for middleware and adapters, adversarial tests for concurrent access patterns, corrupted input handling, and TOCTOU race conditions. (`tests/adversarial/`)

Reproducibility: All benchmarks are deterministic (no network calls, no random seeds beyond jitter). See `benchmarks/README.md` for reproduction instructions.

---

8. Discussion

8.1 Design Decisions

Chain-level versus call-level containment. The central design decision in VERONICA is to enforce limits at the chain boundary rather than per-call. This requires a persistent context object (`ExecutionContext`) that outlives individual calls, which in turn requires lifecycle management (context managers, middleware). The alternative – per-call limits – is simpler to implement but cannot prevent compositional failures because each individual call is within its local budget.

No override mechanism. `PolicyPipeline` has no override or exception path. Any policy denial terminates evaluation. This is intentional: safety policies should not be overridable by lower-priority code. Applications requiring conditional exceptions should configure narrower policies rather than override existing ones.

Cooperative cancellation. G5 (wall-clock timeout) provides cooperative rather than preemptive cancellation. This trades completeness (in-flight operations may complete after timeout) for safety (no thread interruption, no resource leak from killed threads). Long-running operations can poll `CancellationToken.is_cancelled` for finer-grained cancellation.

Redis for distribution. The choice of Redis for distributed state is pragmatic: Redis `INCRBYFLOAT` and Lua `eval()` provide the atomic operations required for G1 and G4 in distributed mode. Alternative backends (PostgreSQL, DynamoDB) would require explicit compare-and-swap loops; Redis provides native atomicity at lower latency.

8.2 Limitations

1. **Cooperative cancellation.** G5 applies only to new operations. In-flight operations at timeout time may complete before observing cancellation.
2. **`RetryContainer` must use `ExecutionContext` for G3.** Standalone `RetryContainer` provides only local bounds; the chain-level ceiling requires an active `ExecutionContext`.
3. **Redis availability for G1 distributed.** When Redis is unavailable and `fallback_on_error=True`, the `LocalBudgetBackend` fallback provides only process-local bounds, not cross-process bounds.
4. **Cost accuracy.** G1 bounds accumulated reported costs, not actual costs. If the LLM provider reports costs inaccurately, the bound applies to reported values.
5. **Agent framework integration depth.** Framework adapters wrap the outermost callable boundary; agent-internal retry logic (e.g., LangChain's `tenacity` integration at the chain level) may not be captured by `RetryContainer` without explicit wrapping.

8.3 Future Work

Since the original v2.0 paper, the following have been implemented: async-native budget enforcement (`AsyncBudgetBackend`, v2.3); reserve-commit-rollback budget protocol (v2.0); OpenTelemetry metric exports for containment events (v2.4); declarative YAML/JSON policy with hot-reload (v2.1); multi-tenant hierarchical budget pools (v2.3); A2A trust boundary with per-agent policy routing (v2.7); and memory governance hooks (v3.4). The v4.0 roadmap targets cross-process federation with cryptographic budget grants.

---

9. Conclusion

VERONICA provides a runtime containment layer that addresses the compositional resource amplification failures inherent in LLM agent systems. By enforcing hard limits at the chain level rather than the call level, VERONICA provides guarantees that per-call policies cannot: cost bounds that hold across nested retry layers (G1), step limits that apply across tool recursion (G2), retry budgets that span agent topologies (G3), and circuit breakers that isolate failures across process boundaries (G4).

The four core primitives – `BudgetEnforcer`, `AgentStepGuard`, `CircuitBreaker`, and `RetryContainer` – are composable via `PolicyPipeline` and require no modification to existing agent logic. Empirical evaluation shows 53–90% operation reduction versus uncontained baselines, with full-pipeline overhead of 11.43 microseconds per call, less than 0.002% of typical LLM API latency. The implementation is available as `veronica-core` on PyPI (v3.4.2, 4844 tests, 94% coverage).

References

[Harrison et al., 2022] Harrison Chase. LangChain. GitHub, 2022. https://github.com/langchain-ai/langchain

[Wu et al., 2023] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.

[LangChain, 2024] LangChain. LangGraph: Build Resilient Language Agents as Graphs. GitHub, 2024. https://github.com/langchain-ai/langgraph

[Moura, 2024] Joao Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. GitHub, 2024. https://github.com/crewAIInc/crewAI

[Nygard, 2007] Michael T. Nygard. Release It!: Design and Deploy Production-Ready Software. Pragmatic Bookshelf, 2007. ISBN 978-0978739218.

Appendix A: API Quick Reference

```python
from veronica_core.containment import ExecutionContext, ExecutionConfig, WrapOptions
from veronica_core import BudgetEnforcer, AgentStepGuard, CircuitBreaker, RetryContainer
from veronica_core.runtime_policy import PolicyPipeline, PolicyContext

# Compose a policy pipeline
pipeline = PolicyPipeline([
    BudgetEnforcer(limit_usd=10.0),
    AgentStepGuard(max_steps=25),
    CircuitBreaker(failure_threshold=5, recovery_timeout=60.0),
])

# Configure chain limits
config = ExecutionConfig(
    max_cost_usd=1.0,
    max_steps=50,
    max_retries_total=10,
    timeout_ms=30_000,
)

# Execute with containment
with ExecutionContext(config=config, pipeline=pipeline) as ctx:
    for step in agent_steps():
        decision = ctx.wrap_llm_call(
            fn=lambda: client.chat(messages=step.messages),
            options=WrapOptions(
                operation_name="agent_step",
                cost_estimate_hint=step.estimated_cost,
            ),
        )
        if decision == Decision.HALT:
```

```
            result = ctx.get_partial_result()
            break
```

Appendix B: Benchmark Reproduction

```
# Install
pip install veronica-core==2.0.0

# Or from source
git clone https://github.com/amabito/veronica-core
cd veronica-core && pip install -e .

# Run all benchmarks
python benchmarks/bench_retry_amplification.py
python benchmarks/bench_recursive_tools.py
python benchmarks/bench_multi_agent_loop.py
python benchmarks/bench_websocket_runaway.py
```

All benchmark scripts are self-contained with stub LLM/tool implementations and produce JSON output for programmatic consumption. No API keys, no network access, no random seeds required for the benchmark results reported in Section 6.

Appendix C: Test Suite Structure

```
tests/
  unit/
    test_budget.py                  # BudgetEnforcer, including NaN/Inf/negative
    test_agent_guard.py             # AgentStepGuard, increment-before-check
    test_circuit_breaker.py         # State machine, HALF_OPEN slot constraint
    test_retry.py                   # RetryContainer, jitter, backoff
    test_execution_context.py       # ExecutionContext, WrapOptions, snapshots
    test_runtime_policy.py          # PolicyPipeline, PolicyContext, PolicyDecision
  integration/
    test_middleware.py              # ASGI/WSGI middleware, WebSocket
    test_adapters_langchain.py      # LangChain adapter
    test_adapters_ag2.py            # AG2 capability adapter
    test_distributed.py             # Redis backend (fakeredis + lupa)
  adversarial/
    test_concurrent_budget.py       # Race conditions on BudgetEnforcer
    test_concurrent_circuit.py      # TOCTOU on CircuitBreaker HALF_OPEN
    test_corrupted_input.py         # NaN, Inf, garbage strings in Redis state
    test_partial_failure.py         # Backend unavailable mid-operation
```